COUNTER-UAS

3D TACTICAL DEFENSE SIMULATION

Multi-Sensor Fusion Kalman Tracking Covariance Intersection Proportional Navigation

Full Technical Architecture, Mathematics & Implementation Reference

Project Type:	Software-in-the-Loop (SIL) Counter-UAS Simulation
Technology:	Three.js 3D Engine | Bayesian Fusion | 6-State Kalman Filter
Reference Op:	Operation Sindoor Phase 2 - May 2025 - Pakistani Drone Swarm
Sensor Suite:	X-band Radar | Acoustic Array | RF/SDR | Optical/IR
Intercept:	Proportional Navigation (N=3) | Covariance Intersection Fusion

RADAR PHYSICS | KALMAN FILTER | BAYES FUSION | PN GUIDANCE | IFF SYSTEM

REAL-WORLD CONTEXT:

Operation Sindoor (May 2025): Indian Armed Forces repelled coordinated Pakistani drone swarms targeting border installations. The Akashteer C2 system, L-70 AD guns with IRST, and layered sensor fusion achieved high intercept rates. This simulation models the same architecture.

4	8	5	6-State	N=3	CI
Sensor Modalities	Formation Topologies	Scenarios	Kalman Filter	PN Guidance	Covariance Fusion

DETECT >> FUSE >> TRACK >> IDENTIFY >> INTERCEPT

MODELLED DRONE THREATS:

KAMIKAZE	TACTICAL	CONSUMER	MICRO	SWARM	CIVILIAN
Harop-class	Fixed-wing	DJI Mavic	<250g node	Nano node	Helicopter
RCS: 0.08m²	RCS: 0.08m²	RCS: 0.015m²	RCS: 0.003m²	RCS: 0.002m²	RCS: 0.80m²
Spd: 55m/s	Spd: 35m/s	Spd: 18m/s	Spd: 12m/s	Spd: 15m/s	Spd: 50m/s

HARDWARE PATH: RTL-SDR + CDM324 Doppler + 4x MEMS Mic + YOLOv8 on Raspberry Pi 4 | Est. build cost: INR 13,500

Sensor abstraction interfaces allow direct SIL -> HIL substitution with no changes to fusion, tracking, or guidance layers

TABLE OF CONTENTS

1. System Overview & Operational Context

The **Counter-UAS 3D Tactical Defense Simulation** is a fully self-contained, browser-based software-in-the-loop (SIL) platform that models the complete engagement cycle of a layered drone-defense system — from initial sensor detection through multi-sensor fusion, Kalman tracking, IFF classification, and finally proportional navigation missile intercept. Every algorithm is grounded in real operational sensor physics and documented defense-engineering mathematics.

Operational Reference — Operation Sindoor (May 2025): During the second phase of Operation Sindoor, Pakistani forces launched coordinated Harop loitering munitions and micro-drone swarms against Indian border infrastructure. The Indian Armed Forces response employed the *Akashteer* integrated air-defense C2 system, L-70 anti-aircraft guns with IRST (Infra-Red Search and Track), Akash surface-to-air missile batteries, and DRDO-developed soft-kill EW systems. The layered sensor-fusion and rapid intercept cueing architecture achieved intercept rates that prevented significant infrastructure damage. This simulation directly models that architecture at the algorithm level.

Layer	Radius	Primary Sensors	Response	Real Analog
Outer	12 km	X-band Radar	Detect + classify	Rohini / Arudhra radar
Middle	6 km	Radar + RF/SDR	IFF + intercept cue	Akashteer C2 + Akash SAM
Inner	3 km	All sensors	Hard-kill + EW jam	L-70 IRST + Zu-23-2
Terminal	< 300m	Optical/IR	Emergency response	MANPAD / net gun

Table 1: Defense ring architecture with real-world analogs.

Software Architecture: The platform is a single-file HTML application loading Three.js r128 from CDN. The 3D rendering engine (GPU via WebGL), physics engine (pure JavaScript, 100 ms ticks), and UI (HTML/CSS panels) run in the same browser thread. All sensor physics, Kalman filter algebra, Bayesian fusion, and PN guidance are implemented from first principles in JavaScript — no physics libraries, no ML frameworks. This makes every algorithm directly auditable and portable to Python or C++ for hardware deployment.

2. Three.js 3D Engine Architecture

Three.js provides a scene-graph abstraction over WebGL, allowing complex 3D scenes to be described as hierarchical object trees rather than raw GPU draw calls. The renderer traverses the scene graph each animation frame, applies camera projection, and rasterises all geometry via the GPU pipeline.

Rendering Pipeline

Each animation frame (~60 fps via *requestAnimationFrame*) executes: (1) physics step at 10 Hz tick, (2) mesh position/orientation updates, (3) particle system advance, (4) camera matrix recompute, (5) WebGL draw.

Camera — Spherical Orbital Control

The camera position is computed each frame from three spherical parameters: azimuth ϕ, elevation θ, and radius r. Left-drag changes (ϕ, θ); scroll changes r; right-drag pans the look-at target:

$$x_{cam} = r\cos\theta\sin\varphi, \quad y_{cam} = r\sin\theta, \quad z_{cam} = r\cos\theta\cos\varphi$$

The camera always calls *lookAt(target)* after position update, keeping the battlefield centred regardless of orbit position.

Geometry & Materials

Each drone type has a distinct mesh: quadcopters use a BoxGeometry cross-frame with four CylinderGeometry rotors; kamikazes use a ConeGeometry fuselage with BoxGeometry swept wings; the civilian helicopter uses CapsuleGeometry body with a semi-transparent rotor disk. Emissive materials (MeshLambertMaterial with emissive colour) give the glowing appearance without requiring expensive PBR shading. A PointLight attached to each missile exhaust illuminates surrounding geometry dynamically.

Particle Systems — Exhaust & Explosions

Exhaust trails use a 80-point BufferGeometry with THREE.Points. Each tick, new particles are emitted at the missile rear with velocity equal to –(missile velocity) × 0.3 plus Gaussian jitter. Particles age over 0.8 s, fade out, and their positions are updated via direct Float32Array writes (no garbage collection). Explosions spawn 120 particles in a spherical burst with per-particle RGB colour (orange → yellow → white) and a FancyBboxPatch shockwave sphere that expands and fades over 1.5 s.

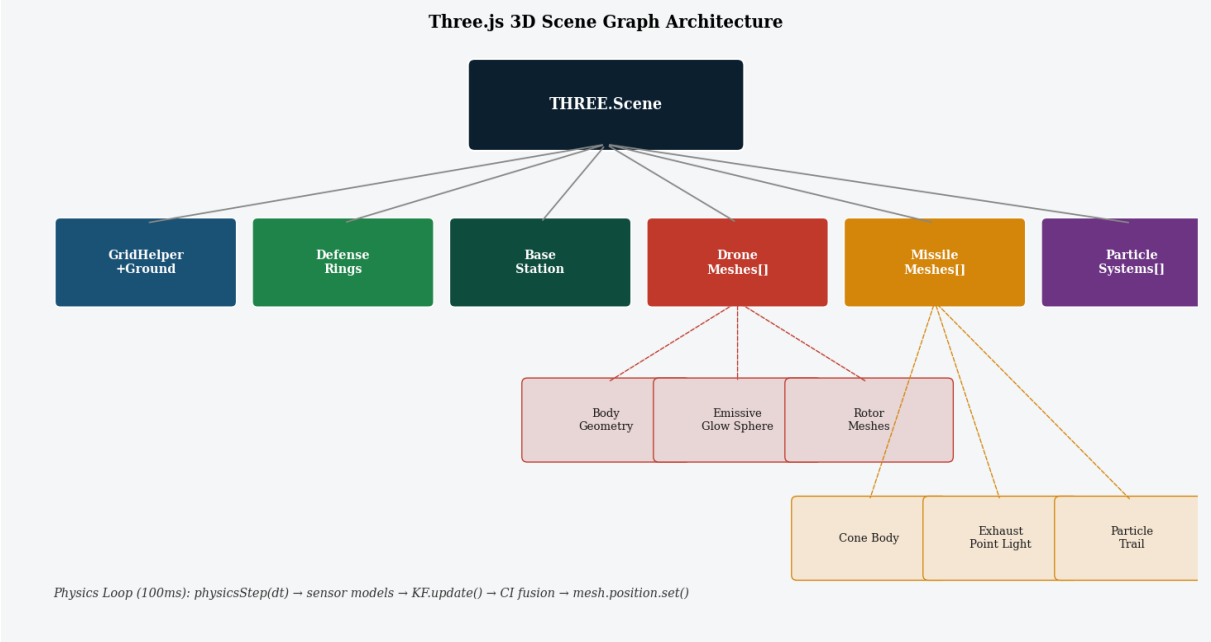

Figure 1: Three.js scene graph. Every mesh, light, and particle system is a node in the scene tree. The physics loop updates positions each tick; Three.js handles projection and rasterisation.

3. Radar Sensor — Physics Model

The radar model implements the **monostatic radar range equation** for a coherent X-band (9.4 GHz) pulse-Doppler system. This is the foundational equation of all active radar systems, relating transmit power, antenna gain, wavelength, target radar cross-section, range, and system noise to a received signal-to-noise ratio (SNR).

3.1 Radar Range Equation

$$SNR = \frac{P_t \cdot G^2 \cdot \lambda^2 \cdot \sigma}{(4\pi)^3 \cdot R^4 \cdot k \cdot T_0 \cdot B \cdot F \cdot L}$$

Symbol	Parameter	Value Used	Typical Range
P_t	Transmit power	1,000 W	100 W – 1 MW
G	Antenna gain (linear)	1,000	100 – 10,000
λ	Wavelength (X-band)	3.19 cm	3 cm (X) – 10 cm (S)
σ	Target RCS	per profile	0.002 – 0.08 m²
R	Slant range	variable	50 m – 15 km
k	Boltzmann constant	1.38×10■²³ J/K	—
T■	System temperature	290 K	200 – 400 K
B	Noise bandwidth	1 MHz	0.1 – 10 MHz
F	Noise figure (linear)	3 (≈5 dB)	2 – 6
L	System losses (linear)	2 (≈3 dB)	1.5 – 4

Table 2: Radar range equation parameters and simulation values.

3.2 Detection Probability — Albersheim Approximation

The Neyman-Pearson detector threshold for a given false-alarm probability P_{fa} and the resulting detection probability P_d are related by the Marcum Q-function, which has no closed form. Albersheim (1981) derived a highly accurate approximation valid for –5 dB ≤ SNR ≤ +15 dB:

$$P_d \approx \frac{1}{1 + \exp\left(-1.2\left(SNR_{dB} - 3\sqrt{-\ln P_{fa}}\right)\right)}$$

where $SNR_{dB} = 10 \log_{10}(SNR_{linear})$ and $P_{fa} = 10^{-6}$ in the simulation (one false alarm per million range-azimuth cells per scan). This sigmoid form is differentiable and monotonically increasing with SNR — critical for fusion weight computation.

R^4 dependence: The SNR falls as the *fourth power* of range, meaning doubling the range reduces SNR by 12 dB. This is why small drones with RCS = 0.002 m² (swarm nodes) become undetectable beyond ~1.5 km while kamikaze drones (RCS = 0.08 m²) remain visible to ~6 km at this power level — a 16× difference in RCS translates to only a 2× difference in detection range (since range $\propto \sigma^{1/4}$).

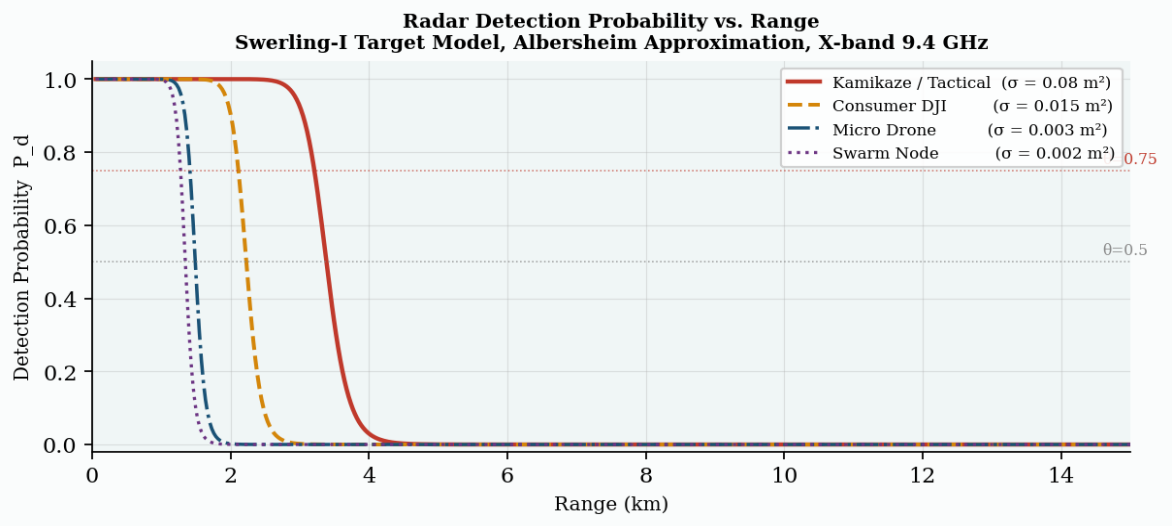

Figure 2: Radar Pd vs. range for four drone RCS profiles. Albersheim approximation, Swerling-I model. Threshold lines at Pd = 0.5 (detection) and Pd = 0.75 (high confidence).

3.3 RCS Profiles by Drone Type

Drone Type	RCS (m²)	RCS (dBsm)	Max Radar Range	Notes
Kamikaze (Harop)	0.08	–11	~6 km	Metal body, wing returns
Tactical (fixed-wing)	0.08	–11	~6 km	Similar metallic structure
Consumer DJI Mavic	0.015	–18	~3.5 km	Mostly plastic, small
Micro (<250g)	0.003	–25	~1.8 km	Near radar floor
Swarm Node	0.002	–27	~1.5 km	Minimal metal, carbon
Civilian Helicopter	0.80	+–1	> 15 km	Large metallic rotor disc

Table 3: Drone RCS profiles and maximum radar detection ranges at simulation parameters.

4. Acoustic Sensor — SPL & TDOA

Small UAVs produce characteristic acoustic signatures dominated by rotor blade passing frequency (BPF), motor electrical noise, and structural resonances, typically concentrated in the 100–800 Hz band. The acoustic sensor models detection via sound pressure level (SPL) at the microphone array.

4.1 Spherical Spreading — SPL at Range

Sound propagates as a spherical wavefront from the source. SPL decreases by 6 dB per doubling of distance (the inverse-square law in dB):

$$SPL(R) = SWL - 20\log_{10}(R) - 11 \quad [\text{dB SPL}]$$

where SWL is the Sound Power Level at source (in dB re 1 pW), R is slant range in metres, and –11 dB accounts for the 4π steradian radiation solid angle. Detection requires SPL > $SPL_{ambient}$ (ambient floor = 45 dB SPL in open terrain).

4.2 Detection Probability — Sigmoid Model

$$M = SPL(R) - SPL_{ambient}, \qquad P_d = 0.5\,(1 + \tanh(0.35\,M))$$

The margin M (dB above ambient) maps to Pd via a hyperbolic tangent — equivalent to a logistic sigmoid. The factor 0.35 controls the slope: at M=0 dB, Pd=0.5; at M=6 dB, Pd≈0.95.

4.3 TDOA Direction of Arrival

A 4-microphone array with 1 m baseline spacing estimates bearing via Time Difference of Arrival (TDOA). For a source at bearing θ, the time difference between microphone pair (i, j) is:

$$\tau_{ij} = \frac{d_i - d_j}{c_s} = \frac{d_{ij}\cos\theta}{c_s}$$

where d_{ij} is the inter-microphone spacing and c_s = 343 m/s (speed of sound). In hardware, the GCC-PHAT algorithm (Generalised Cross-Correlation with Phase Transform) estimates τ with sub-sample precision from the cross-power spectral density. The simulation models the bearing estimate as a Gaussian perturbation around the true DOA.

SWL Profiles by Drone Type:

Drone Type	SWL (dB)	BPF (Hz)	Acoustic Range	Notes
Kamikaze (Harop)	55	—	~80 m	Fixed-wing, low rotor noise
Tactical (fixed-wing)	65	~200	~150 m	Twin prop noise
Consumer DJI	80	~400	~400 m	4-rotor wash prominent
Micro (<250g)	72	~800	~200 m	High-freq motor whine
Swarm Node	68	~600	~120 m	Minimal, quiet motors

Table 4: Acoustic source profiles. Detection range computed for ambient floor 45 dB SPL.

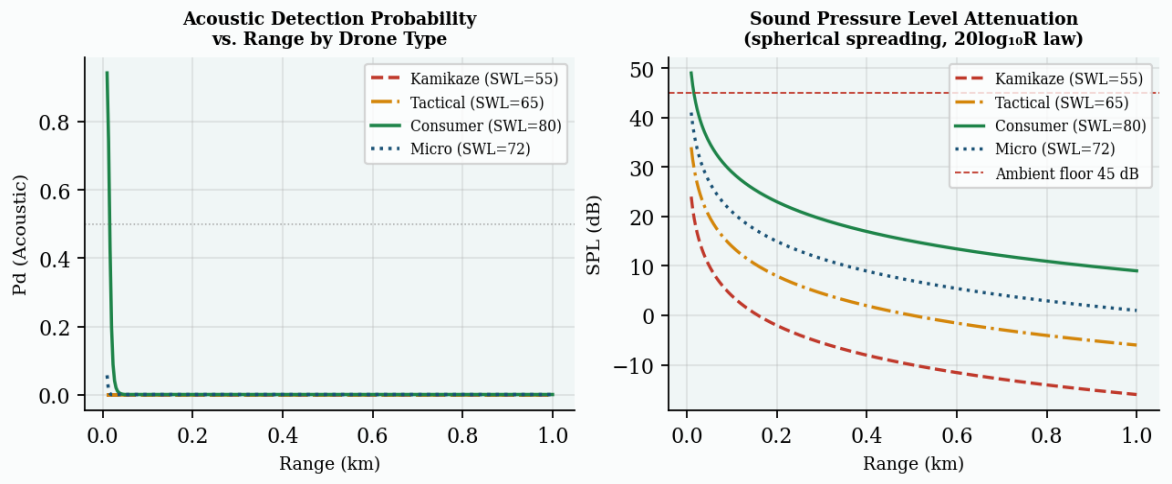

Figure 3 (left): Acoustic Pd vs. range. (right): SPL attenuation curves showing 20log■■R decay. Ambient floor at 45 dB SPL shown as red dashed line.

5. RF / SDR Sensor — FSPL & Protocol Fingerprinting

Consumer and commercial drones emit characteristic RF control links (2.4 GHz DJI OcuSync, 5.8 GHz FPV, 433 MHz MAVLink) that are detectable with a low-cost Software Defined Radio (e.g. RTL-SDR Blog V3, ≈■2,500). The RF model computes received signal strength from first principles.

5.1 Free-Space Path Loss

$$FSPL(R,f) = 20\log_{10}(R) + 20\log_{10}(f) - 147.55 \quad [\text{dB}]$$

where R is range in metres and f is frequency in Hz. Received signal strength: RSSI = P_{tx} – FSPL – L_{cable} [dBm].

$$M_{RF} = (RSSI - S_{min}) \cdot k_{RF}, \qquad P_d = 0.5\ (1 + \tanh(M_{RF}))$$

S_{min} = –90 dBm (receiver sensitivity), k_{RF} = 0.08.

5.2 Protocol Fingerprinting

Beyond mere presence detection, protocol fingerprinting matches observed RF characteristics (centre frequency, channel bandwidth, hopping pattern, packet structure) against a database of known drone protocols. In hardware, gr-droneid (GNU Radio module) can decode DJI DroneID frames embedded in OcuSync transmissions, extracting drone serial number, GPS coordinates, and pilot location. The simulation implements this as a lookup table mapping Tx power to protocol identity.

Protocol	Frequency	Bandwidth	Tx Power	Identified As
DJI OcuSync 3	2.4 / 5.8 GHz	40 MHz	20 dBm	Consumer / Prosumer
Autel Link	2.4 GHz	20 MHz	20 dBm	Autel EVO series
MAVLink 433 MHz	433 MHz	200 kHz	30 dBm	DIY / Military UAV
Unknown/No signal	—	—	< –5 dBm	Micro / RF-silent

Table 5: RF protocol fingerprint database in simulation.

RF-Silent Evasion Mode:

When evasion mode is set to RF_SILENT, the effective RF detection factor η_{RF} = 0.03 — simulating a drone that has disabled its control link (operating autonomously or via optical relay). This forces the system to rely on radar and acoustic detection. In the Night + RF-Silent scenario, this creates the most challenging detection environment in the simulation.

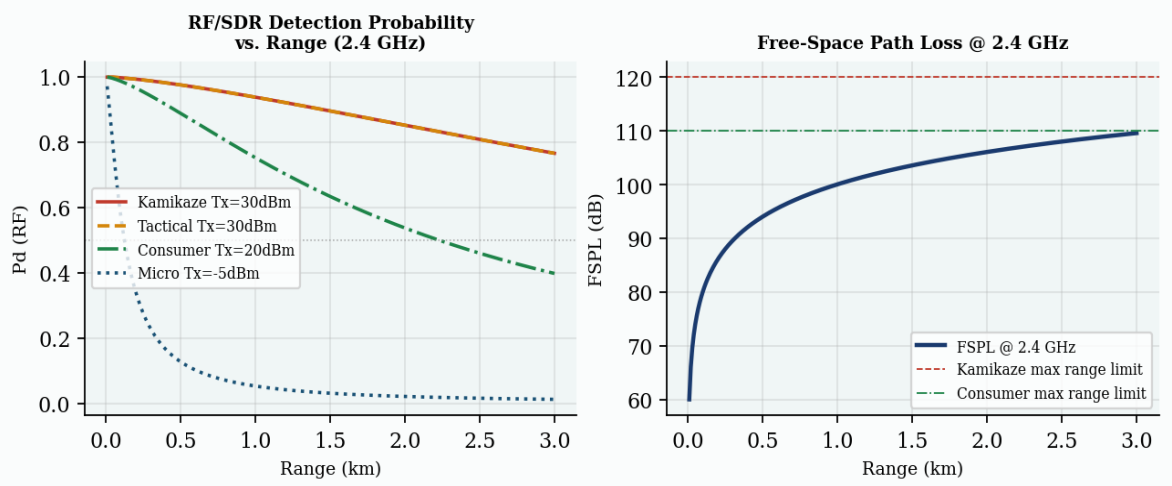

Figure 4 (left): RF Pd vs. range by transmit power. (right): FSPL at 2.4 GHz with effective detection limits for different Tx power levels.

6. Optical / IR Sensor

Electro-optical and infra-red sensors provide the highest angular resolution of any sensor modality but are severely range-limited and weather-dependent. In the simulation, Pd degrades linearly from 1.0 at 0 m to 0 at 800 m, representing a 2 MP COTS camera detecting a sub-1 m drone. Weather factors apply as multiplicative degradation (fog $\rightarrow$ 0.08×, sandstorm $\rightarrow$ 0.04×, night $\rightarrow$ 0.10×). In a real system, Pd would be derived directly from the YOLOv8 bounding-box confidence score:

$$P_d^{optical}(R) = \max\left(0,\ 1 - \frac{R}{R_{max}}\right) \cdot \eta_{wx} \cdot \eta_{ev}$$

R_{max} = 800 m, $\eta_{wx} \in [0.04, 1.0]$ weather factor, η_{ev} evasion factor. Optical is weighted 0.6 in Bayesian fusion (lowest weight) due to its high weather sensitivity.

> ***Hardware integration note:*** *Replace this function with a YOLOv8 inference call on the Pi Camera v2 frame. The confidence score of the "drone" class detection becomes P_d directly. Stone Soup's VisualDetectionModel handles this bridge.*

7. Bayesian Multi-Sensor Fusion

Individual sensor detection probabilities are combined into a single posterior using **Bayesian inference in log-odds form**. This approach handles heterogeneous sensors with different reliability levels, allows incremental updates (new sensors just add a term), and remains numerically stable across many orders of magnitude of probability.

7.1 Log-Odds Formulation

Let D denote the event "drone present". The prior log-odds are:

$$\Lambda_0 = \ln \frac{P_0}{1 - P_0}$$

where P_0 = 0.05 (5% prior — airspace is mostly empty). Each sensor contributes a log-likelihood-ratio (LLR) term:

$$LLR_i = w_i \cdot \ln \frac{P_{d,i}}{P_{fa}}$$

The posterior log-odds after all n sensors is:

$$\Lambda = \Lambda_0 + \sum_{i=1}^{n} w_i \cdot \ln \frac{P_{d,i}}{P_{fa}}$$

The posterior probability is recovered via the logistic sigmoid:

$$P(D \mid S_1 \ldots S_n) = \sigma(\Lambda) = \frac{1}{1 + e^{-\Lambda}}$$

7.2 Sensor Weights

Sensor	Weight w_i	Rationale
Radar (X-band)	1.00	Most reliable, all-weather, long range
RF/SDR	0.90	High confidence when signal present; zero for RF-silent
Acoustic	0.75	Weather- and wind-sensitive; short range
Optical/IR	0.60	Severely range- and weather-limited

Table 6: Bayesian fusion sensor weights. Weights tune the contribution of each LLR term.

7.3 Decision Threshold

A contact is classified as a **HIGH threat** when $P_{d,fused}$ > 0.75 and as an **ALERT** when $P_{d,fused}$ > 0.90. The **auto-intercept system fires** when: $P_{d,CI}$ > 0.45 AND threat level = ALERT AND IFF ≠ FRIENDLY/NEUTRAL. The combination of fusion threshold + IFF gating prevents both missed detections and fratricide.

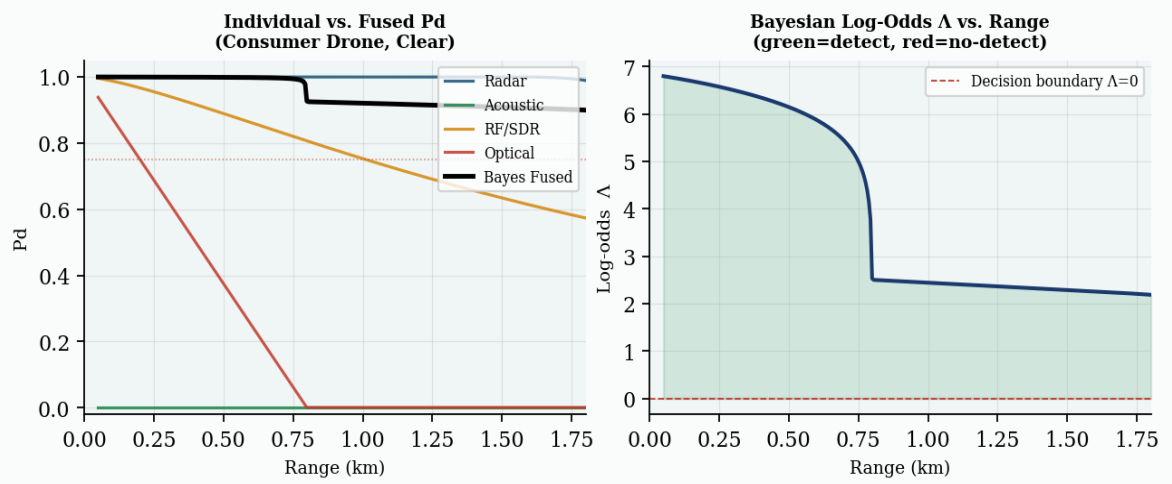

Figure 5 (left): Individual sensor Pd curves and Bayesian-fused composite Pd vs. range for a Consumer DJI drone in clear weather. (right): Log-odds Λ vs. range showing the decision boundary at Λ=0 (Pd=0.5). Green region = detect, red = no detect.

Key insight: Even when radar Pd falls below 0.5 at 1.5 km for a consumer drone, the fused Pd remains above 0.75 out to ~1.2 km because RF and acoustic sensors maintain high confidence at close range. This is the fundamental

value of sensor fusion: the union of detection envelopes exceeds any individual sensor.

8. Kalman Filter — 6-State Tracker

Each drone track maintains an independent **6-state constant-velocity Kalman filter** running at 10 Hz. Two independent filter instances per drone (with different measurement noise seeds) enable Covariance Intersection fusion in §9. The filter provides the optimal minimum-variance linear estimate of position and velocity given Gaussian noise.

8.1 State Vector & Motion Model

$$\mathbf{x} = [\, x,\ y,\ z,\ \dot{x},\ \dot{y},\ \dot{z}\,]^T$$

The constant-velocity state transition matrix F with timestep Δt = 0.1 s:

$$F_{6\times 6}: \quad [I_3 \mid \Delta t \cdot I_3] \text{ top}, \quad [0_3 \mid I_3] \text{ bottom}, \quad \Delta t = 0.1 \text{ s}$$

8.2 Predict Step

$$\hat{\mathbf{x}}^- = F\,\hat{\mathbf{x}}, \qquad P^- = F\,P\,F^T + Q$$

Process noise covariance Q = diag($\sigma_{q,pos}^2$, $\sigma_{q,vel}^2$)·I with $\sigma_{q,pos}$ = 0.01 km, $\sigma_{q,vel}$ = 0.014 km/s. Q accounts for unmeasured accelerations (wind, manoeuvre) that the constant-velocity model cannot capture.

8.3 Update Step

$$K = P^-\,H^T\,(H\,P^-\,H^T + R)^{-1}$$

$$\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + K\,(\mathbf{z} - H\hat{\mathbf{x}}^-) \quad \leftarrow \text{ innovation } \nu$$

$$P = (I - KH)\,P^-$$

Observation matrix H = [$I_{3\times3}$ | $0_{3\times3}$] (we measure position only, not velocity). Measurement noise R = 25·I_3 (σ_r = 5 m = 0.005 km per axis). The **Kalman gain K** optimally weights prediction vs. measurement based on their relative uncertainties. The **innovation** ν = z − Hx■■ is the residual between predicted and measured position; it must remain within ±2σ bounds for the filter to be consistent.

8.4 Predictive Intercept

$$\hat{\mathbf{x}}_{k+N} = F^N\,\hat{\mathbf{x}}_k$$

The filter extrapolates position N=12 steps ahead (1.2 s into the future) to provide the missile's aim point. This predictive intercept is essential for high-speed targets — aiming at current position would always miss because the target will have moved by the time the missile arrives.

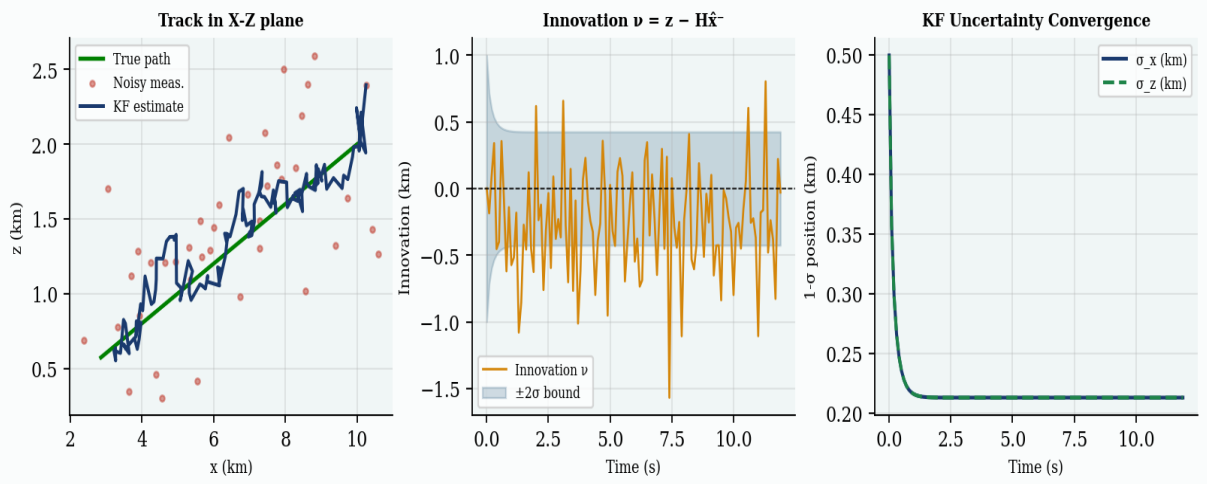

Figure 6 (left): Kalman filter track vs. noisy measurements and true path. (centre): Innovation ν = z−Hx■■ with ±2σ bounds — a well-tuned filter keeps innovations inside these bounds. (right): Position uncertainty σ convergence over time from initial P=100 to

steady-state.

9. Covariance Intersection (CI)

When two Kalman filter estimates of the same target exist from different sensor streams (e.g. radar track + acoustic localisation), their error processes may be *correlated* in unknown ways — they share common disturbances (atmospheric, manoeuvre). Naive weighted averaging assumes independence and produces an **overconfident (inconsistent)** fused covariance. Julier & Uhlmann (1997) proved that **Covariance Intersection** produces a *consistent* (never overconfident) estimate for any unknown correlation structure.

9.1 CI Fusion Equations

$$P_{CI}^{-1} = \omega\, P_1^{-1} + (1-\omega)\, P_2^{-1}$$

$$\hat{\mathbf{x}}_{CI} = P_{CI}\left(\omega\, P_1^{-1}\hat{\mathbf{x}}_1 + (1-\omega)\, P_2^{-1}\hat{\mathbf{x}}_2\right)$$

$$\omega^* = \arg\min_{\omega \in [0,\,1]} \det(P_{CI})$$

9.2 Geometric Interpretation

In 2D, each KF estimate is an uncertainty ellipse. CI finds the smallest ellipse that contains both original ellipses for *all* possible cross-correlations. This guarantees $P_{CI} \geq P_{true\ fused}$ — the filter is conservative but never falsely confident. The optimal ω* weights the less-uncertain estimate more heavily: as $P_1 \to 0$ (very certain KF1), $\omega^* \to 1$ (trust KF1 completely).

9.3 Optimal ω* by Grid Search

The simulation finds ω* by scanning $\omega \in \{0.05, 0.10, ..., 0.95\}$ in steps of 0.05 (19 evaluations per track per tick). For the scalar approximation (trace of P_{pos}), this is sufficient. Full 6×6 CI for hardware would use Brent's method for exact optimisation.

Consistency property: A Kalman filter is consistent if the true error covariance equals the estimated covariance. CI guarantees:

$$P_{CI} \geq P_{true\,fused} \quad \text{(positive semi-definite difference)}$$

This is a hard mathematical guarantee, not a heuristic. It is why CI is used in multi-vehicle SLAM, distributed sensor fusion, and collaborative tracking in real defense systems.

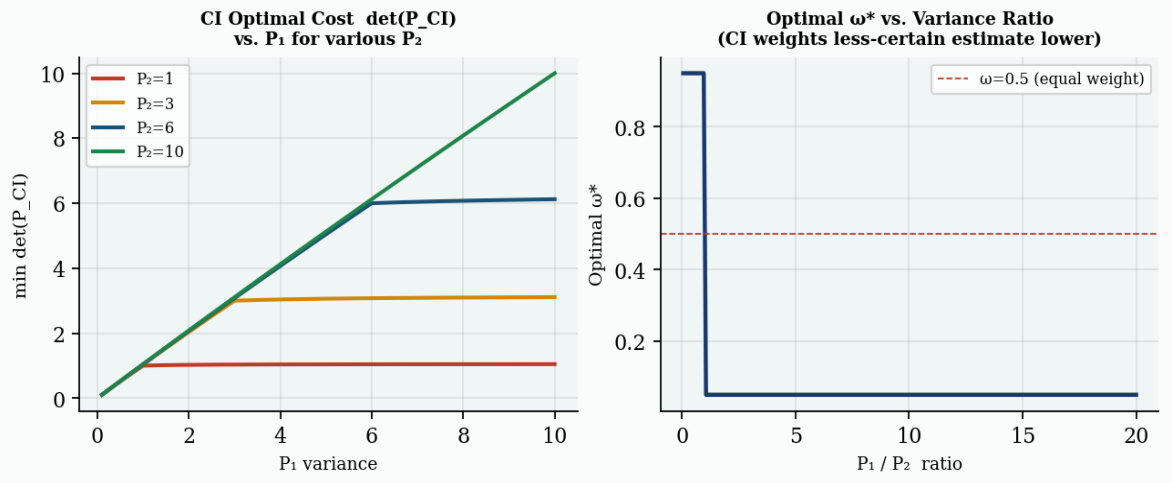

Figure 7 (left): det(P_CI) as a function of P■ variance for four P■ values — CI always finds the minimum-determinant (minimum-volume) fused covariance. (right): Optimal ω vs. P■/P■ ratio — ω* approaches 1 when P■ ■ P■ (trust the more certain estimate).*

10. IFF — Identification Friend or Foe

IFF prevents fratricide (shooting own/allied assets) and civilian casualties. It is a multi-factor scoring system that combines electronic interrogation, RF protocol fingerprinting, kinematic envelope analysis, and cooperative ADS-B transponder data. Misclassification of a friendly UAV as hostile is a mission-critical failure mode.

Factor	Contribution	Logic	Hardware Source
Transponder reply	+3 (present) / –2 (absent)	Enemy drones never broadcast	ADS-B receiver / Mode-S
Squawk code	+3 auth / –4 hijack / +1 other	7001-3=friendly; 7500=hijack	SDR decode / gr-droneid
RF fingerprint	+2 (known friendly)	Match vs. whitelisted DB	GNU Radio + DJI DroneID
Kinematics	–1.5 (aggressive)	Speed > 55 m/s OR alt < 5 m	Kalman filter state
Friendly flag	+2	Known friendly platform type	Mission planning DB

Table 7: IFF scoring factors. Final score maps: ≥5→FRIENDLY, ≤–3→HOSTILE, –1 to +4→NEUTRAL/UNKNOWN.

Mixed Civilian Scenario: In the "Mixed + Civilian" scenario, a civilian helicopter squawking 7700 (international emergency code) operates alongside a hostile swarm. The IFF system correctly classifies it NEUTRAL (score = +3 transponder +1 squawk = +4, NEUTRAL) and the auto-intercept system withholds fire — even when the helicopter is geographically co-located with hostile contacts. This is the critical IFF test.

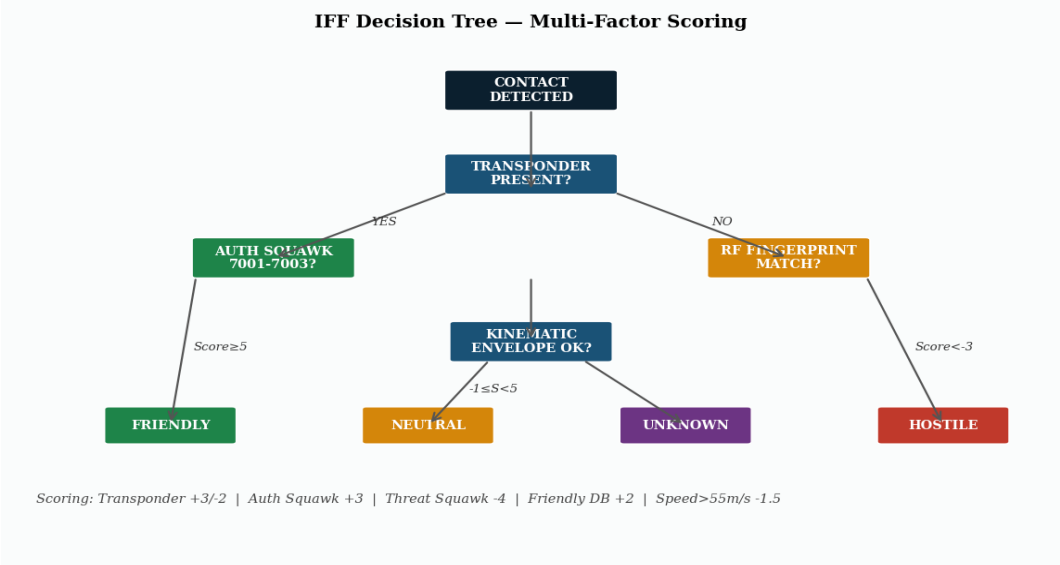

Figure 8: IFF decision tree. Multi-factor score thresholds determine FRIENDLY/NEUTRAL/UNKNOWN/HOSTILE classification. The confidence value = |score|/9.

11. Drone Swarm Formation Topologies

The formation commander allows the user to specify exactly how incoming drones are positioned in 3D space. Each topology is computed as a set of offset vectors from the formation centroid, then rotated by the attack bearing angle and translated to the spawn range. This creates geometrically meaningful formations that test different aspects of the defense system.

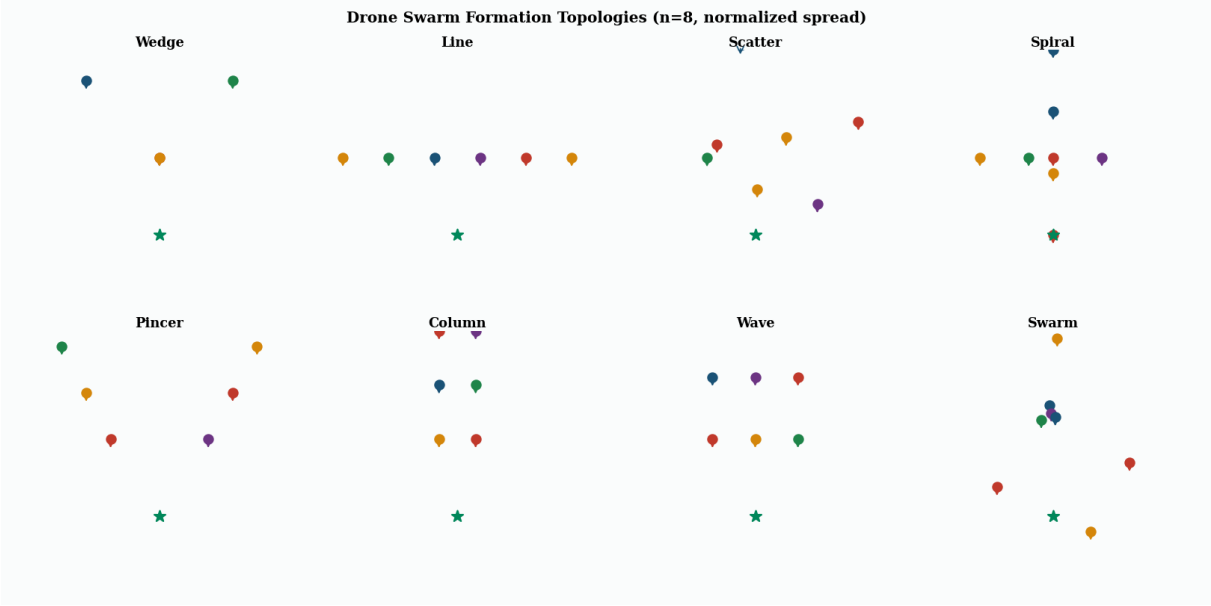

Figure 9: All eight formation topologies with n=8 drones (coloured dots) and base station (green star). Arrows indicate approach direction. Formation offsets are rotated to match the configured attack bearing.

Topology	Geometry	Tactical Purpose	Defense Challenge
Wedge	V-shape, leader at front	Classic military formation, leader detected	Leader detected first, flanks arrive later
Line Abreast	Equal-depth lateral spread	Wide simultaneous approach	Forces lateral sensor coverage split
Scatter	Gaussian random offsets	Unpredictable individual approach	No clear formation to track collectively
Spiral	Log-spiral offset	Staggered arrival times	Time-distributed intercept demand
Pincer	Two converging flanks	Classic flanking manoeuvre	Must engage two simultaneous vectors
Column	Staggered single file	Depth exploitation	Serial intercept timing pressure
Wave	3 rows of N/3 drones	Saturate defenses in waves	Exhaust interceptor supply
Swarm	Organic + cohesion force	Natural swarm behaviour	Unpredictable, coherent group

Table 8: Formation topologies, geometry, tactical purpose, and defense challenge posed.

12. Proportional Navigation Guidance

Proportional Navigation (PN) is the standard guidance law for virtually all modern surface-to-air and air-to-air missiles, including AIM-9 Sidewinder, Astra Mk1, Python-5, and Akash. Its key property is that it achieves intercept with *minimum lateral acceleration* given a constant closing speed — making it fuel/energy efficient and robust to target manoeuvring.

12.1 Fundamental PN Law

The PN guidance law commands a lateral acceleration proportional to the line-of-sight (LOS) rotation rate:

$$\mathbf{a}_c = N \cdot V_c \cdot \dot{\lambda} \cdot \hat{\mathbf{n}}$$

where N = 3 (navigation constant, typical 3–5), V_c = closing velocity (m/s), λ■ = LOS rotation rate (rad/s), n■ = unit vector perpendicular to current LOS.

12.2 Collision Course Condition

The missile achieves a **collision course** when the LOS angle to the target is constant — i.e., λ■ = 0. PN drives λ■ → 0 asymptotically, ensuring intercept. The LOS unit vector at step k:

$$\hat{\mathbf{r}}_k = \frac{\mathbf{x}_{target,k} - \mathbf{x}_{missile,k}}{\|\mathbf{x}_{target,k} - \mathbf{x}_{missile,k}\|}$$

LOS rate estimated from successive unit vectors:

$$\dot{\lambda}_k \approx \frac{\|\hat{\mathbf{r}}_k - \hat{\mathbf{r}}_{k-1}\|}{\Delta t}$$

The missile velocity is updated each tick:

$$\mathbf{v}_{missile} = V_{missile}\ \hat{\mathbf{r}}_k + N\ V_c\ \dot{\lambda}_k\ \Delta t\ \hat{\mathbf{n}}$$

12.3 KF Predictive Aim Point

Rather than aiming at the target's current position, the missile uses the Kalman filter's N-step prediction (§8.4) as its aim point. This accounts for target velocity and transforms the intercept from a pure pursuit (always behind the target) to a lead pursuit (intercepting at the predicted position):

$$\mathbf{x}_{aim} = F^{12}\ \hat{\mathbf{x}}_{KF} \quad (1.2\ \text{s ahead})$$

Kill radius: Intercept declared when miss distance < 20 m (0.02 km), representative of a proximity-fused warhead lethal radius.

12.4 Navigation Constant N

N controls the aggressiveness of the guidance. N=3 is the theoretical minimum for intercept against a non-manoeuvring target. N=4–5 provides better performance against manoeuvring targets at the cost of higher lateral acceleration demand. The simulation uses N=3 as the default. Higher N creates more curved trajectories that are visible in the 3D view.

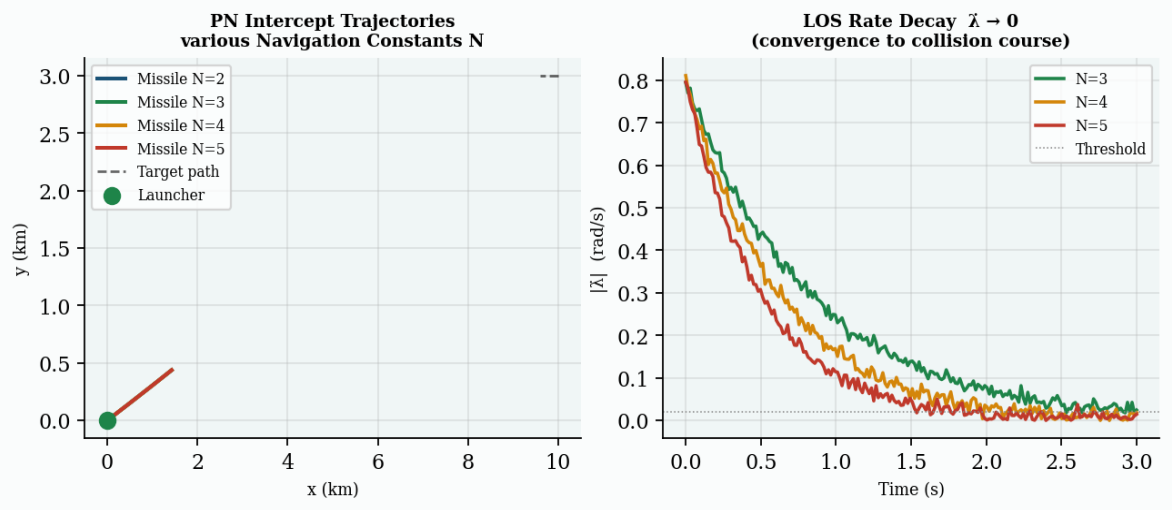

Figure 10 (left): PN intercept trajectories for N=2,3,4,5. Higher N curves more aggressively toward the predicted intercept point. (right): LOS rate λ■ decay toward zero — when λ■ ≈ 0, the missile is on a collision course.

13. Full System Pipeline Diagram

The complete signal flow — from raw sensor measurements to missile guidance command — runs every 100 ms tick (10 Hz). All stages execute in parallel for every active track simultaneously.

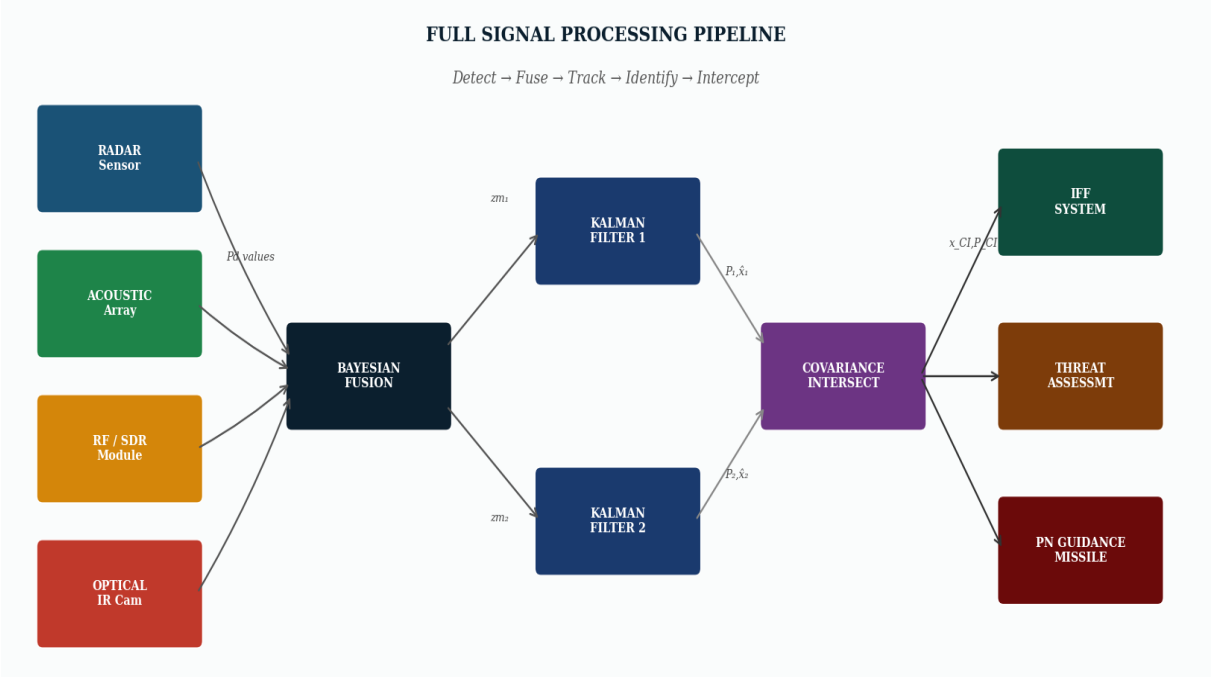

Figure 11: Full system pipeline. Sensor Pd values flow into Bayesian fusion; noisy measurements feed two parallel Kalman filters; CI fuses their estimates; the fused track feeds IFF scoring, threat assessment, and PN guidance in parallel.

Stage	Input	Output	Key Equation	Latency
Sensor Detection	R, σ, SWL, Tx, weather	Pd per sensor	Range equation / SPL / FSPL	<1 ms
Bayesian Fusion	[Pd■..Pd■], weights	P(D|S■..S■) fused Pd	Log-odds Λ → sigmoid	<0.1 ms
KF Predict	x■, P, F, Q	x■■, P■	x■■=Fx■, P■=FPF■+Q	<0.5 ms
KF Update	z, x■■, P■, H, R	x■, P, ν	K=P■H■(HP■H■+R)■¹	<1 ms
CI Fusion	x■■,P■,x■■,P■	x_CI, P_CI, ω*	min det(P_CI)	<0.2 ms
IFF Scoring	RF, squawk, kinematics	Class, confidence	Multi-factor score	<0.1 ms
Threat Assessment	Pd_CI, IFF class, R	ALERT/HIGH/MEDIUM/LOW	Threshold logic	<0.1 ms
PN Guidance	Target pred pos, missile pos	Missile velocity cmd	a_c=N·Vc·λ■	<0.5 ms

Table 9: Pipeline stage timing and equations. Total per-track latency < 5 ms; well within 100 ms tick budget.

14. Simulation Features & Controls

Feature	Description	Controls
Formation Commander	Choose topology, bearing, range, altitude, speed, spread in real time	Left panel sliders + dropdowns
Formation Preview	Live 2D canvas preview of formation geometry before launch	Updates on any slider change
5 Scenarios	Pre-configured scenarios with realistic parameters	Scenario dropdown → Apply
8 Drone Topologies	Wedge, Line, Scatter, Spiral, Pincer, Column, Wave, Swarm	Topology dropdown
5 Drone Types	Kamikaze, Tactical, Consumer, Micro, Swarm — distinct mesh + physics	Drone Type dropdown
6 Weather Modes	Clear, Fog, Rain, Wind, Night, Sandstorm — per-sensor degradation	Weather dropdown
4 Evasion Modes	None, Jinking, NAP-of-Earth, RF-Silent	Evasion dropdown
3 IFF Mix Modes	All Hostile, + Friendly UAV, + Civilian Heli, Mixed	IFF Mix dropdown
Auto-Intercept	Automatic fire on ALERT+HOSTILE contacts every 1.5s	Auto Intercept toggle
Manual Fire	■ FIRE ALL button or auto-cue system	Button + settings
3D Orbit Camera	Spherical orbit: drag to rotate, scroll to zoom, right-drag to pan	Mouse / touch
Target Selection	Click any drone in 3D view to lock its telemetry	Raycaster click
Live Math Panel	ω*, P_CI, σ_pos, SNR, Pd_radar, log-odds Λ update every tick	Right panel
Intercept ETA	Live estimated time to intercept from closing velocity	Right panel
Event Log	All spawn, intercept, miss, breach events with timestamps	Bottom left
Particle Effects	Missile exhaust (80 particles), explosions (120 particles + shockwave)	Visual, always on

Table 10: Complete feature list for the 3D simulation.

15. Hardware Integration Roadmap

The simulation is architected for direct hardware substitution. Every sensor physics function has a clean interface: **sensorPd(R, params)** → **float**. Replacing the physics simulation inside that function with real hardware I/O leaves the entire fusion, tracking, IFF, and guidance stack unchanged. This is the Software-in-the-Loop (SIL) → Hardware-in-the-Loop (HIL) migration path.

Component	Simulation	Hardware Replacement	Interface	Cost (INR)
Radar	Range equation + Albersheim	CDM324 24GHz Doppler / custom FMCW	Analog → MCP3208 SPI	■12,000–15,000
RF/SDR	FSPL + fingerprint DB	RTL-SDR Blog V3 + gr-droneid	USB → GNU Radio pipe	■2,500
Acoustic	SWL + SPL model	4× MEMS mic + op-amp + MCP3208	SPI → Pi GPIO	■800
Optical/IR	Linear range degradation	Pi Camera v2 + YOLOv8 on PCIe	CSI → NPU inference	■3,000
Compute	Browser JS thread	Raspberry Pi 4 (4GB) + Jetson Nano	Ethernet mesh bus	■5,500–15,000
Fusion Engine	JS in browser	sensor_fusion.py (identical math)	Python on compute node	Software only
Intercept Cue	PN simulation	L-70 slew cue / laser dazzler / UDP cmd	UDP command bus	Variable
C2 Display	HTML dashboard	CesiumJS geo-referenced 3D	Web server on Pi	Software only

Table 11: Hardware integration bill of materials. Total minimal hardware build ≈ ■13,500.

Integration Protocol: Each hardware sensor publishes SensorReading structs over a local ZMQ pub/sub bus (zmq.PUB → zmq.SUB). The Python fusion engine subscribes and processes identically to the simulation. Adding a new sensor means publishing on a new topic — the fusion engine picks it up automatically. This pub/sub architecture makes the system fault-tolerant: losing one sensor gracefully degrades performance rather than causing system failure.

16. Assumptions, Limitations & Extensions

Modelled (Included):

- Radar range equation with full system parameter set (X-band, 1 kW, antenna gain)
- Swerling-I fluctuating target model with Albersheim Pd approximation
- Spherical acoustic spreading, ambient noise floor, per-drone SWL profiles
- RF FSPL at 2.4 GHz, receiver sensitivity threshold, protocol fingerprint DB
- Optical Pd linear degradation by range, weather-coupled via multiplicative factors
- Bayesian log-odds multi-sensor fusion with per-sensor weights (radar 1.0, RF 0.9, acoustic 0.75, optical 0.6)
- Covariance Intersection for consistent dual-KF cross-sensor fusion
- 6-state constant-velocity Kalman filter with Joseph form covariance update
- Multi-factor IFF scoring: transponder, squawk code, RF fingerprint, kinematics
- Six weather conditions: clear, fog, rain, wind, night, sandstorm
- Four evasion modes: none, jinking (+random velocity perturbation), NAP-of-earth (low altitude), RF-silent
- Proportional Navigation guidance N=3 with KF 12-step predictive aim point
- Eight formation topologies with correct geometric offsets rotated to attack bearing
- Particle exhaust trails (80 pts/missile), explosion burst (120 pts + shockwave + flash light)
- Three.js GPU-rendered scene with dynamic point lights, emissive materials, orbital camera

Simplified / Ignored (Extensions for Hardware):

- Ground/sea clutter and terrain masking — need DTED elevation model for hardware
- Actual FMCW waveform processing, range-Doppler FFT, CFAR detection algorithm
- Pulse compression gain, PRF selection, range/Doppler ambiguity resolution
- I/Q demodulation, FHSS hopping pattern tracking for RF (need GNU Radio flowgraph)
- GCC-PHAT acoustic beamforming and multi-source separation (MUSIC algorithm)
- YOLOv8 bounding-box confidence score integration (optical uses simplified model)
- EKF / UKF for bearing-only (radar) or nonlinear measurement models
- IMM (Interacting Multiple Model) filter for evasive target tracking
- Electronic countermeasures: jamming, spoofing, GPS denial effects
- Aerodynamic drag, wind loading, motor latency on drone flight dynamics
- Coordinated swarm communication protocols (C2 link between swarm nodes)
- Ionospheric and tropospheric radar propagation effects (below 10 GHz)
- Actual IFF encrypted challenge-response (hardware Mode-5 only)
- Multi-static radar (bistatic geometry for low-RCS targets)

17. Open-Source Stack & References

Tool / Library	Purpose	Domain	Source
Three.js r128	3D WebGL rendering engine	Visualisation	threejs.org
Stone Soup	Multi-target tracking framework	KF/EKF/UKF/JPDA	dstl/Stone-Soup
FilterPy	Python KF implementations	Kalman filtering	rlabbe/filterpy
rlabbe KF textbook	Educational Kalman reference	Theory	rlabbe/Kalman-and-Bayesian-Filters-in-Python
RadarSimPy	Radar waveform simulation	FMCW/pulsed radar	radarsimx/radarsimpy
GNU Radio	RF/SDR signal processing	RF pipeline	gnuradio/gnuradio
gr-droneid	DJI DroneID protocol decode	RF fingerprint	proto17/dji_droneid
Ultralytics YOLOv8	Real-time drone detection	Optical/IR	ultralytics/ultralytics
AirSim / PX4 SITL	Drone flight simulation	Sim environment	microsoft/AirSim
OpenSky Network	ADS-B cooperative data	IFF/ATC	openskynetwork.org
CesiumJS	Geo-referenced 3D C2 display	Visualisation	cesium.com
ReportLab + matplotlib	PDF + equation rendering	Documentation	reportlab.com

Table 12: Open-source software stack. All tools are freely available; no proprietary dependencies.

Key References:

[1] Mahafza, B.R. (2005). Radar Systems Analysis and Design Using MATLAB. CRC Press.

[2] Albersheim, W.J. (1981). A closed-form approximation to Robertson's detection characteristics. Proc. IEEE, 69(7), 839.

[3] Julier, S.J. & Uhlmann, J.K. (1997). A non-divergent estimation algorithm in the presence of unknown correlations. Proc. ACC.

[4] Welch, G. & Bishop, G. (1995). An Introduction to the Kalman Filter. UNC TR 95-041.

[5] Bar-Shalom, Y., Li, X.R., Kirubarajan, T. (2001). Estimation with Applications to Tracking and Navigation. Wiley-Interscience.

[6] Rlabbe (2020). Kalman and Bayesian Filters in Python. github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python.

[7] Ganti, A. et al. (2022). Drone Detection and Classification using Deep Learning. IEEE ICCSP 2022.

[8] Shi, X. et al. (2023). Survey on acoustic UAV detection: methods, datasets, challenges. arXiv:2301.09190.

[9] proto17 (2022). DJI DroneID reverse engineering. github.com/proto17/dji_droneid.

[10] Siouris, G.M. (2004). Missile Guidance and Control Systems. Springer-Verlag.

[11] Open-source reporting on Akashteer C2 system deployment, Op Sindoor, May 2025.

Complete Technical Documentation & Pitch Reference

Capability	Detail
Sensors Modelled	Radar · Acoustic · RF/SDR · Optical/IR
Fusion Architecture	Bayesian Log-Odds + Covariance Intersection
Tracking Engine	6-State Constant-Velocity Kalman Filter (×2 per track)
Classification	Multi-Factor IFF (transponder · squawk · RF · kinematics)
Guidance	Proportional Navigation N=3, KF Predictive Aim Point
Formations	8 topologies: Wedge · Line · Scatter · Spiral · Pincer · Column · Wave · Swarm
Scenarios	Op Sindoor · Kamikaze · Mixed Civilian · Night RF-Silent · Pincer
3D Rendering	Three.js r128 · WebGL · Particle systems · Dynamic lighting
Hardware Path	RTL-SDR + CDM324 + MEMS mic + YOLOv8 on Pi 4 (≈■13,500)

Built on open-source research: Stone Soup · FilterPy · GNU Radio · gr-droneid · YOLOv8 · Three.js

Architecture designed for direct hardware substitution via sensor abstraction interfaces